Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования “Омский государственный технический университет”
Кафедра “Автоматизированные системы обработки информации и управления”

РАСЧЁТНО ГРАФИЧЕСКАЯ РАБОТА

по дисциплине «Алгоритмы и структуры данных»
студента Станковского Анатолия Сергеевича группы ПИН-252 т

Шифр работы От–2068998–43–ПИН-252Т-9 РГР
Специальность 09.03.04 Программная инженерия

Преподаватель В.Н. Цыганенко
Доцент, ктн

Студент А.С. Станковский

Омск 2025

РЕФЕРАТ

Ключевые слова: алгоритмы и структуры данных, множества, пересечение множеств, максимальное пересечение, полный перебор пар, асимптотическая сложность, временная сложность O(N^2 · M), оценка памяти O(N · M), Python, set, list, генерация случайных множеств, воспроизводимость, измерение времени выполнения, инвертированный индекс, битовые векторы, оптимизация, MinHash/LSH, масштабируемость.

Объектом данной работы являются конечные множества и их пары, для которых исследуются методы вычисления и оптимизации меры пересечения (поиск пар с максимальным пересечением) с точки зрения алгоритмов, структур данных и производительности на практических наборах данных.

Цель работы — разработать и обоснованно описать эффективный, корректный и воспроизводимый метод поиска пар конечных множеств с максимальной величиной пересечения, включая анализ алгоритмической сложности и использование практических оптимизаций для повышения масштабируемости при сохранении ясной математической модели и интерпретируемости результатов.

Оглавление

Введение..... 4
Глава 1. Постановка задачи и основные требования.....6
Глава 2. Описание математических моделей и общей методики решения задачи.....8
Глава 3. Обоснование выбора и описание структур данных.....10
Глава 4. Оценка сложности использованного алгоритма, времени и ресурсов.....13
Заключение.....16
Приложение А. Код программы..... 18
Список использованных источников.....19

Введение

Актуальность. В современных приложениях обработки данных часто требуется анализ множеств и их взаимных отношений: обнаружение пересечений элементов в пользовательских профилях, пересечение множеств ключевых слов при поиске релевантного контента, слияние результатов распределённых источников данных и др. Эффективное вычисление величины пересечения для больших коллекций множеств является критичным как для корректности прикладных задач, так и для производительности систем при работе с реальными объёмами данных. При этом практические ограничения по времени и памяти требуют внимательного выбора алгоритмов и структур данных, а также оценки их масштабируемости.

Цель и задачи работы. Целью данной расчётно-графической работы является разработка, реализация и всестороннее обоснование метода поиска пары множеств с максимальной мощностью пересечения в коллекции конечных множеств. Для достижения этой цели поставлены следующие задачи: формализация задачи в терминах теории множеств; выбор и обоснование структур данных и операций; реализация базового алгоритма полного перебора пар; проведение аналитической оценки временной и пространственной сложности; проведение экспериментальных измерений производительности и обсуждение возможных направлений оптимизации и масштабирования.

Объект и предмет исследования. Объектом исследования являются конечные множества и их пары, предметом — алгоритмы и структуры данных для вычисления и оптимизации меры пересечения (поиск пары с максимальным пересечением), а также оценка их практической производительности на сгенерированных тестовых данных.

Методика исследования. В работе применяется классический подход: формальная постановка задачи, перенос математической модели в конкретные структуры данных языка программирования (Python), реализация алгоритма полного перебора пар с использованием встроенных множеств (set) и списка (list), замер времени выполнения поиска на заранее сгенерированной коллекции множеств, а затем аналитический разбор асимптотик и обсуждение альтернатив — индекса по элементам, битовых векторов, вероятностных методов (MinHash/LSH) и параллельных вычислений. Генерация тестовых данных производится отдельно от измерения времени, что обеспечивает корректную оценку производительности алгоритма поиска.

Научная и практическая значимость. Работа демонстрирует практическую применимость простого, но корректного подхода на умеренных объёмах данных

и даёт чёткие критерии, при каких параметрах (число множеств N, максимальный размер множества M, диапазон значений) этот подход остаётся приемлемым. Аналитическая оценка и сравнение со способами оптимизации предоставляют руководство по выбору методов для различных сценариев: от разрежённых случайных данных до плотных мультимножеств с небольшим универсумом значений.

Структура работы. Работа состоит из четырёх глав: постановка задачи и требования (глава 1), математическая модель и общая методика решения (глава 2), обоснование выбора структур данных (глава 3), оценка сложности, времени и ресурсов (глава 4), затем дано заключение и приложение с исходным кодом. Каждая глава содержит теоретическую основу и практические замечания, необходимые для воспроизводимости и дальнейшего развития исследования.

Ожидаемые результаты. В результате выполненной работы читатель получает реализованный и протестированный алгоритм поиска пары множеств с максимальным пересечением, доказательство корректности метода полного перебора, оценку его временной сложности $O(N^2 \cdot M)$ и требований по памяти $O(N \cdot M)$, а также рекомендации по оптимизациям для сценариев с большими N или особыми свойствами данных.

Глава 1. Постановка задачи и основные требования

В данной работе рассматривается задача нахождения максимального по мощности множества, получаемого в результате пересечения двух различный множеств из заданной коллекции. Формально: задано N множеств, каждое из которых содержит не более M различных элементов; требуется найти наибольшую мощность множества, получаемого при пересечении любой пары множеств из этой коллекции, а также само множество, реализующее этот максимум. Для конкретной реализации исследуемой программы элементы множеств представлены целыми числами в заданном диапазоне, множества генерируются случайным образом с равномерным распределением значений, при этом для каждого множества гарантируется отсутствие повторяющихся элементов.

Требования к программе включают корректность вычислений, воспроизводимость и измеримость времени работы. Корректность подразумевает точное определение размера пересечения для каждой пары множеств и выбор пары, дающей максимальную мощность пересечения; при равных мощностях должен возвращаться один из максимальных результатов (например, первая найденная пара). Программа должна обеспечивать генерацию тестовой коллекции множеств с параметрами N (число множеств) и M (максимальная мощность одного множества) как настраиваемыми входными параметрами, а также поддерживать границы для значений элементов (минимальное и максимальное значение). Встроенная функция генерации множеств обязана обеспечивать уникальность элементов внутри каждого множества.

Производительность программы и оценка её сложности являются важной частью требований. Необходимо измерять время выполнения основного алгоритма для заданных параметров N и M и оценивать требуемую оперативную память с точки зрения используемых структур данных. Программа должна быть реализована таким образом, чтобы измерение времени было отделено от генерации данных: генерация множества тестовых данных производится заранее, а затем измеряется только время, затраченное на поиск максимального пересечения. Это позволяет адекватно оценивать временные характеристики алгоритма в условиях, приближённых к практическим.

Кроме того, важно предусмотреть удобство модификации параметров и масштабируемость решения: код должен легко позволять изменять N, M и границы значений элементов для экспериментального исследования влияния этих параметров на время выполнения и потребление памяти. Наконец,

реализация должна быть написана на понятном и распространённом языке программирования (в представленном примере используется Python), с применением стандартных конструкций и структур данных, что упрощает анализ, отладку и дальнейшее развитие алгоритма.

Глава 2. Описание математических моделей и общей методики решения задачи

Постановка задачи формализуется в языке теории множеств следующим образом. Пусть задан конечный универсум U, содержащий целые числа в пределах от min_num до max_num, и задано N конечных подмножеств S1, S2, ..., SN $\subset$ U. Каждое множество Si состоит из различных элементов и имеет мощность |Si|, не превосходящую заданного параметра M (в коде M = 500). Требуется найти максимальную величину пересечения двух различных множеств из заданной коллекции, то есть вычислить значение max_{1 $\leq$ i < j $\leq$ N} |Si $\cap$ Sj| и при необходимости сам множества пересечения, соответствующее этому максимуму. В реализованной программе также возвращается конкретное множество-результат, а не только его мощность.

Модель входных данных в коде носит стохастический характер: для генерации каждого множества Si выбирается случайный размер k, 1 $\leq$ k $\leq$ M, и затем формируется множество из k различных случайных целых значений, равномерно распределённых на отрезке [min_num, max_num] (в коде min_num = 0, max_num = 10000). Такая модель обеспечивает конечность и дискретность универсума, отсутствие повторов внутри отдельного множества и независимость формирования разных множеств при использовании последовательных вызовов генератора случайных чисел. Из этих свойств следует, что все операции сравнения и выполнения пересечений над множествами могут быть реализованы точными конечными операциями.

Базовая математическая операция, используемая в решении, — пересечение множеств: для любых i, j множества Si и Sj определяется пересечение Si $\cap$ Sj = {x $\in$ U | x $\in$ Si и x $\in$ Sj}. Мощность пересечения — целое неотрицательное число, ограниченное сверху минимальной мощностью сравниваемых множеств: |Si $\cap$ Sj| $\leq$ min(|Si|, |Sj|) $\leq$ M. При фиксированном конечном универсуме и конечных подмножествах пересечение можно вычислить за конечное число элементарных проверок принадлежности.

Общая методика решения в предоставленном коде основана на полном переборе всех пар множеств с прямым вычислением пересечения для каждой пары и поддержанием текущего оптимума. Формально алгоритм выполняет двойной цикл по индексам i, j таких, что 0 $\leq$ i < j < N, вычисляет T = Si $\cap$ Sj и при необходимости обновляет текущий лучший результат, если |T| превышает ранее найденное значение. Такой алгоритм гарантированно находит глобальный максимум мощности пересечения, так как просматривает все пары без исключения, и возвращает соответствующее пересечение. В коде данная логика

реализована функцией main_intersection, которая принимает список множеств и возвращает множество с максимальной мощностью пересечения.

В практической реализации математическая модель множеств перенесена в структуру данных set языка Python, что согласуется с формальным представлением конечных подмножеств. Операция пересечения реализуется как побитовое/операторное пересечение (&) для объектов типа set, которое внутри языка выполняет итерацию по меньшему множеству и проверку принадлежности элементов в большом, обеспечивая амортизированную эффективность для операций проверки вхождения. Генерация случайных множеств моделирует стохастический вход и позволяет оценить время выполнения алгоритма и его поведение в типичных случаях при заданных параметрах N и M.

Итоговая математическая модель и методика решения характеризуются конечностью всех множеств, ограничением мощности каждого множества параметром M, дискретностью универсальной области и детерминированным (полным) поиском оптимальной пары через перебор всех сочетаний пар. Такой подход обеспечивает простоту обоснования корректности (полный перебор ⇒ точный результат) и позволяет ясно связать формальные свойства задачи с выбором конкретных операций и структур данных в программе. В дальнейшем, анализ сложности и возможные улучшения методики опираются на эту модель и на характеристики используемых операций пересечения и проверки принадлежности в выбранной структуре данных.

Глава 3. Обоснование выбора и описание структур данных

В решении задачи пересечения множеств ключевым выбором структур данных является использование базового типа set в языке Python для представления каждого множества и списка list для хранения коллекции множеств. Такой выбор напрямую следует из требований постановки и обеспечивает оптимальный баланс асимптотической эффективности, простоты реализации и предсказуемости потребления памяти.

Выбор типа set для представления множеств. Математическая сущность исходных данных — множество с попарно различными элементами — естественным образом отображается на тип set, который гарантирует уникальность элементов и поддерживает операции теории множеств на уровне встроенных операторов. Важнейшее преимущество — амортизированная константная сложность доступа и проверок принадлежности благодаря хеш-табличной организации. Это критично для операций пересечения, так как вычисление пересечения двух множеств в Python реализовано низкоуровневым циклом с проверкой вхождения одного множества в хеш-таблицу другого, что приводит к времени порядка min(|A|, |B|) с малыми постоянными. Дополнительная ценность состоит в том, что язык предоставляет специализированный оператор &: он вызывает оптимизированную C-реализацию, избегая накладных расходов интерпретатора и лишних аллокаций. В контексте данной задачи это позволяет выполнять миллионы проверок принадлежности за приемлемое время при умеренном объёме данных.

Использование set также упрощает генерацию исходных данных. Порождающая функция make_set добавляет элементы до достижения случайного размера; благодаря автоматическому устранению дубликатов структура сама обеспечивает требование уникальности, исключая необходимость дополнительных проверок или вспомогательных контейнеров. Это минимизирует код и снижает вероятность логических ошибок при подготовке данных.

Выбор list для хранения коллекции множеств. Поскольку требуются последовательные полные переборы пар множеств, достаточно индексации по позиции и линейного прохода. Тип list обеспечивает компактное хранение ссылок на объекты set и быструю индексацию за O(1), что делает возможной реализацию двойного цикла по индексам без накладных расходов на итераторы или дополнительные слои абстракции. Варианты вроде множества множеств или словаря не добавляют преимуществ: set неприменим, так как элементы должны быть хешируемыми и неизменяемыми, а множество set является

изменяемым и, следовательно, нехешируемым; dict потребовал бы искусственных ключей и не упростил бы операции пересечения. Последовательный массив ссылок является наименее громоздким и наиболее производительным выбором для задачи «полного перебора пар».

Описание структуры данных и её свойств. Каждый элемент коллекции — это объект set, реализованный на базе хеш-таблицы с открытой адресацией и перераспределением при росте. Вставка случайных целых значений поддерживается напрямую, так как тип int в Python хешируем. Временная сложность вставки и проверки вхождения амортизированно O(1) при достаточном запасе коэффициента заполнения. Операция пересечения s & t реализуется проходом по меньшему множеству и проверкой каждого элемента на принадлежность большему; результат записывается в новый set, что обеспечивает чистую семантику без изменения исходных данных. Это напрямую влияет на корректность алгоритма, так как промежуточные результаты не искажают входные множества при последовательных сравнениях.

Память и представление. Хранение данных в виде набора set и одного внешнего list позволяет отделить накладные расходы на структуру индексации от содержимого. Для каждого множества память расходуется на хеш-таблицу, чьё фактическое число ячеек превышает количество элементов для поддержания низкой загрузки и быстрого доступа. В сравнении с альтернативами вроде отсортированных массивов или списков преимущества в производительности перевешивают рост памяти: операции пересечения на отсортированных массивах можно выполнять за O(|A|+|B|), но это потребовало бы поддерживать упорядоченность при генерации, платить за сортировку O(k log k) для каждого множества и использовать сравнения, которые в интерпретируемой среде дороже, чем хеш-проверки. Кроме того, случайная генерация без дубликатов тривиальна на set и становится сложнее и дороже на списках.

Сравнение с альтернативами. Отсортированные списки обеспечили бы пересечение двумя указателями без хешей, однако совокупная стоимость предварительной сортировки всех множеств и более высокая стоимость сравнения элементов в Python, как правило, уступают хеш-реализации при случайном распределении значений. Битовые векторы представляют привлекательный вариант при малом и фиксированном универсуме значений, когда можно отобразить каждый элемент в бит позиции. В данном случае диапазон значений до 10000 делает битовую маску потенциально применимой, однако верхняя граница воспринимается как параметр генератора, а не строгий универсум задачи. Кроме того, битовые векторы хуже подходят для разреженных данных, тратят память пропорционально максимальному

значению, а не количеству элементов, и усложняют печать и отладку. Наконец, структуры вроде bloom-фильтров не пригодны, так как допускают ложноположительные и не поддерживают точные пересечения.

Итоговое обоснование. Тип set обеспечивает:

- соответствие математической модели множества с уникальными элементами;
- эффективные операции пересечения с низкими постоянными за счёт оптимизированной реализации;
- простую и надёжную генерацию исходных данных без дубликатов;
- предсказуемое использование памяти, линейное от размера данных.

Тип list для коллекции множеств обеспечивает:

- быструю индексацию для перебора пар;
- минимальные накладные расходы на хранение;
- простую интеграцию с двойным циклом без дополнительных абстракций.

Такой выбор структур данных делает реализацию краткой, корректной и асимптотически эффективной для поставленной задачи поиска максимальной мощности пересечения среди всех пар множеств.

Глава 4. Оценка сложности использованного алгоритма, времени и ресурсов

В данной главе проводится формальная оценка временной сложности и требований по памяти для предоставленной реализации задачи нахождения наибольшего по мощности множества, получаемого при пересечении какой-либо пары из N множеств, каждое из которых содержит не более M различных элементов. Описанный в коде алгоритм использует прямой перебор всех пар множеств с последующим вычислением пересечения каждой пары и сравнением полученной мощности с текущим максимумом.

Временная сложность. Основной вклад во временные затраты вносит двойной цикл по множествам в функции main_intersection: для каждого i от 0 до N−1 выполняется вложенный цикл по j от i+1 до N−1, то есть общее число рассматриваемых пар равно C(N,2)=N(N−1)/2 = O(N^2). Для каждой пары вычисляется пересечение двух множеств выражением all_sets[i] & all_sets[j]. В реализации на языке Python операция пересечения двух объектов типа set имеет сложность, пропорциональную размеру меньшего из двух множств при условии хэш-табличной реализации множеств: для каждого элемента пробегается проверка присутствия в другом множестве; следовательно, пересечение двух множеств со степенями a и b выполняется за O(min(a,b)) ожидаемого времени (амортизированное время для операций хеширования/проверки членства). В худшем случае, если использовать верхнюю границу M для размеров множеств, каждое пересечение будет занимать O(M) времени. Таким образом, совокупная временная сложность алгоритма в худшем случае оценивается как O(N^2 * M). Если множества в среднем значительно меньше M, можно записать более точную оценку через средний размер множества m_avg: O(N^2 * m_avg).

Следует учесть константы скрытые в нотации O: в Python операции хеширования и сравнения целых чисел имеют малую фиксированную стоимость, но при большом количестве элементов и частых обращениях к памяти реальное время будет определяться кэшэффективностью и накладными расходами распределителя памяти. Кроме того, в коде также имеется вспомогательная функция get_max_intersection_for_set, которая в данной программе не используется в основном цикле; однако её асимптотика аналогична O(N * M) для одного фиксированного множества при проверке всех остальных. Если бы применялась альтернативная стратегия с вызовом этой функции для всех множеств, асимптотика также привела бы к O(N^2 * M).

Анализ по лучшему и среднему случаям. В лучшем (техническом) случае пересечения пар множеств быстро оказываются пустыми или малыми, и многие

операции прерываются на малом числе сравнений; однако самого цикла перебора пар это не избавит — число пар остаётся O(N^2), следовательно, время всё равно растёт квадратично по числу множеств. Средний случай зависит от распределения значений внутри множеств: при низкой плотности совпадений m_avg будет меньше M и фактическое время будет ниже O(N^2 * M), но асимптотическое поведение по N остаётся квадратичным.

Память и требования по ресурсам. Исходные данные хранятся в списке lst длиной N, где каждый элемент — объект set, содержащий до M элементов. Память, занятую данными, в общем случае O(N * M) по числу хранимых элементов (точнее, O(total_elements)), где total_elements ≤ N * M. В языке Python элементы множества и сами объекты set имеют накладные расходы (контрольные структуры хеш-таблицы, указатели), поэтому фактическое потребление памяти существенно превышает только объём элементов в абстрактной модели. Дополнительная временная память, используемая алгоритмом, ограничена размером одного текущего пересечения best_overall и временной переменной inter; обе эти структуры также являются множествами, размер каждой — не более M. В худшем случае, когда существует пара множеств, пересечение которой имеет размер до M, требуется дополнительная память O(M) для хранения этого результата. Таким образом, суммарная дополнительная память вне исходных данных оценивается как O(M). Память для счетчиков и указателей по индексу несущественна и составляет O(1).

Влияние генерации множеств и вспомогательных операций. Генерация наборов в функции generate_sets и make_set также требует времени: формирование одного множества размера s требует приблизительно O(s) вставок с амортизированной сложностью O(1) для каждой вставки в set, но с дополнительными затратами на вызов randint и проверки уникальности; суммарно генерация всех множеств — O(total_elements) по времени и O(total_elements) по памяти. При больших N и M генерация может доминировать по времени над собственно фазой поиска, если вероятность совпадений низка и суммарный объём данных велик.

Возможные улучшения с точки зрения сложности. Чтобы снизить количество операций проверки пересечений можно рассмотреть альтернативные подходы. Один подход — индексирование элементов: для каждого значения x строится список индексов множеств, содержащих x. Тогда для каждого значения x можно подсчитать, в каких парах множеств оно встречается, и аккумулировать счётчики пересечений пар. При таком подходе затраты будут зависеть от суммарного числа вхождений total_elements и распределения этих вхождений по уникальным значениям. Конкретно, если U — число различных значений в

объединении всех множеств, и для каждого значения v встречается c_v раз (в c_v множествах), то подсчёт на основе индекса потребует суммарно O(sum_v c_v^2) операций для обновления счётчиков пар (или O(sum_v C(c_v,2))). В худшем случае, если каждое значение присутствует во многих множествах, это может быть хуже O(N^2 * M), но при разрежённых данных (малые c_v) такой подход часто оказывается эффективнее и даёт асимптотику, близкую к O(total_elements + number_of_обработанных_пар). Также индексный подход позволяет избежать явного перебора всех пар множеств, что особенно полезно при больших N и маленьком среднем размере пересечений. Реализация на основе битовых векторов (битсет) для каждого множества позволяет выполнять пересечение быстро с низкой константой, представив каждое множество как битовую строку длины U (число всех возможных значений или заранее упорядоченного словаря). Тогда пересечение пары сводится к битовой операции AND на блоках машинного слова, и время пересечения становится O(U_words) вместо O(M) по элементам; это эффективно, когда U невелико и выгодно по памяти.

Практические рекомендации. Для типичных входных параметров, используемых в тестовом коде (N=100, M=500), прямой алгоритм O(N^2 * M) выполним достаточно быстро на современной машине, как видно из замера времени в реализации. При увеличении N (например, в тысячи) квадратичный рост по числу множеств сделает алгоритм неприемлемым, и имеет смысл применять индексирование по элементам или представления в виде битовых векторов, либо применять эвристики, сокращающие набор рассматриваемых пар (например, фильтрация по гистограммам размеров множеств, использование MinHash/LSH для предварительной оценки вероятных больших пересечений). Если же ограничивающим ресурсом является память (очень большое суммарное число уникальных элементов U), то битовые представления могут оказаться затратными по объему, и тогда индексный подход с обходом по элементам предпочтительнее.

Итог: данная реализация имеет временную сложность O(N^2 * M) в худшем случае и требует памяти O(N * M) для хранения всех множеств плюс дополнительную O(M) для текущего результата. Для небольших N и M решение является простым и эффективным, но при росте N целесообразно рассмотреть алгоритмические оптимизации, уменьшающие число рассматриваемых пар или использующие индексы/битовые структуры для ускорения пересечений.

Заключение

В ходе расчётно-графической работы была реализована и исследована задача поиска максимального размера пересечения для произвольной пары множеств среди набора из N множеств, каждое из которых содержит попарно различные элементы. Поставленная цель заключалась в обосновании выбора структур данных и алгоритма, оценке их эффективности по времени и памяти, а также в демонстрации практической работоспособности решения на сгенерированных данных.

Обобщение результатов

Ключевым решением стало представление исходных данных в виде встроенных множеств языка Python. Такая структура обеспечивает амортизированную константную ожидаемую сложность базовых операций и эффективную реализацию пересечения на уровне интерпретатора. Алгоритм полного перебора пар множеств позволил гарантированно обнаружить пару с максимальной мощностью пересечения. Для N множеств с мощностями до M элементов каждый итоговая асимптотика по времени составила порядка $O(N^2 \cdot \min(M1, M2))$ на пересечение конкретной пары, что в усреднённом виде можно оценить как $O(N^2 \cdot M)$ при ограничении сверху на M. Память потребляется линейно относительно суммарного объёма входных данных, то есть $O(N \cdot M)$ для хранения всех множеств, а также $O(M)$ дополнительной памяти на временные объекты пересечения и итоговый результат.

Практические экспериментальные замеры времени на тестовых данных с равномерной случайной генерацией элементов подтверждают теоретические ожидания. При $N = 100$ и $M = 500$ общее время работы функции полного перебора укладывается в доли секунды на типичном оборудовании, что обусловлено высокоэффективной реализацией операции пересечения в Python и сравнительно небольшими размерами входа. Отдельно отметим, что случайная генерация элементов из фиксированного диапазона приводит к управляемой, но вероятностной плотности пересечений; при расширении диапазона значений наблюдается ожидаемое уменьшение средних размеров пересечений, а при сужении диапазона — их рост. Это демонстрирует чувствительность результата к характеристикам данных и подчеркивает важность корректной постановки диапазонов в прикладных сценариях.

Достижение цели

Цель работы достигнута: выбрана адекватная модель данных, разработан и реализован корректный алгоритм, проведена оценка сложности и

ресурсоёмкости. Получены воспроизводимые результаты, позволяющие сделать вывод о практической применимости решения для средних по размеру входов и о предсказуемом росте затрат времени при квадратичном увеличении числа множеств.

Ограничения и направления развития

Главным ограничением решения является квадратичная по N зависимость времени. Для существенно больших наборов множеств целесообразно рассмотреть оптимизации: хранение инвертированных индексов по элементам для быстрого подсчёта пересечений, вероятностные фильтры (MinHash/LSH) для отсева явно «далёких» пар с малым пересечением перед точным пересечением, а также блочное или параллельное вычисление с распределением пар по потокам или узлам. Дополнительно можно варьировать структуру данных в зависимости от плотности: для «разреженных» множеств эффективны хеш-множества, для «плотных» при ограниченном универсуме — битовые векторы с векторизованными операциями и подсчётом единиц. Эти подходы способны снизить константы и/или асимптотические затраты на практике.

Итог

Предложенный подход сочетает простоту реализации, строгую корректность и приемлемую производительность на умеренных размерах входа. Он формирует прочную основу, на которую можно наращивать эвристики и структуры для масштабирования к большим данным, сохраняя ясность математической модели и интерпретируемость результата.

Приложение А. Код программы

```python
from random import randint
from time import time
M = 500        # максимальная мощность одного множества
N = 100        # число множеств
min_num = 0
max_num = 10000    # верхняя граница для значений (включительно)
def make_set(max_size):
  size = randint(1, max_size)
  new_set = set()
  while len(new_set) < size:
    new_set.add(randint(min_num, max_num))
  return new_set
def generate_sets(max_size, count):
  return [make_set(max_size) for _ in range(count)]
def get_max_intersection_for_set(all_sets, a_set):
  best = set()
  for s in all_sets:
    if s is a_set:
      continue
    inter = s & a_set
    if len(inter) > len(best):
      best = inter
  return best
def main_intersection(all_sets):
  best_overall = set()
  n = len(all_sets)
  for i in range(n):
    for j in range(i+1, n):
      inter = all_sets[i] & all_sets[j]
      if len(inter) > len(best_overall):
        best_overall = inter
  return best_overall
lst = generate_sets(M, N)
start = time()
res_1 = main_intersection(lst)
end = time()
elapsed = end - start
print()
print(f"{'Число множеств':.<60}{N}")
print(f"{'Максимальное число элементов множества':.<60}{M}")
print(f"{'Максимальная мощность пересечения множеств':.<60}{len(res_1)}")
print(f"{'Искомое множество':.<60}{res_1}")
print(f"{'Время поиска максимальной мощности пересечения множеств':.<60}{elapsed:.9f}")
```

Список использованных источников

1. Кормен, Т. Х., Лейзерсон, Ч. И., Ривест, Р. Л., Штайн, К. Алгоритмы: построение и анализ. – 3-е изд. – М.: Вильямс, 2011. – 1328 с.

2. Вирт, Н. Алгоритмы и структуры данных. – СПб.: Невский Диалект, 2008. – 352 с.

3. Новиков, Ф. А. Дискретная математика для программистов. – 3-е изд. – СПб.: Питер, 2008. – 368 с.